

by a random word (to reduce the discrepancy between pretraining and fine-tuning, since the model will not see <mask> tokens during fine-tuning), and a 10% chance of being left alone (to bias the model toward the correct answer).

Next sentence prediction (NSP)

The model is trained to predict whether two sentences are consecutive or not. For example, it should predict that “The dog sleeps” and “It snores loudly” are consecutive sentences, while “The dog sleeps” and “The Earth orbits the Sun” are not consecutive. Later research showed that NSP was not as important as was initially thought, so it was dropped in most later architectures.

The model is trained on these two tasks simultaneously (see [Figure 16-11](#)). For the NSP task, the authors inserted a class token (<CLS>) at the start of every input, and the corresponding output token represents the model’s prediction: sentence B follows sentence A, or it does not. The two input sentences are concatenated, separated only by a special separation token (<SEP>), and they are fed as input to the model. To help the model know which sentence each input token belongs to, a *segment embedding* is added on top of each token’s positional embeddings: there are just two possible segment embeddings, one for sentence A, and one for sentence B. For the MLM task, some input words are masked (as we just saw) and the model tries to predict what those words were. The loss is only computed on the NSP prediction and the masked tokens, not on the unmasked ones.

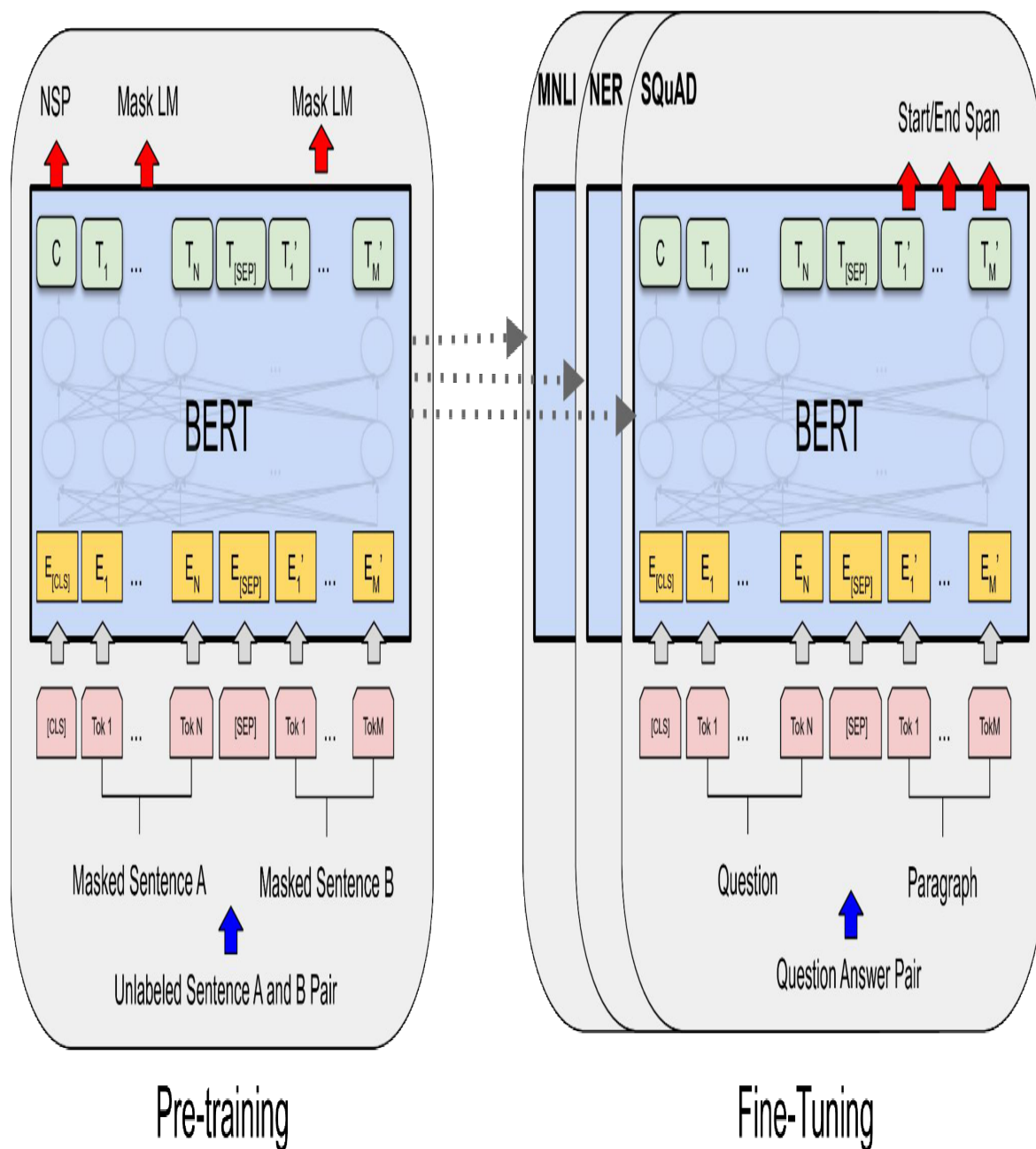


Figure 16-11. BERT training and fine-tuning process³¹

After this unsupervised pretraining phase on a very large corpus of text, the model is then fine-tuned on many different tasks, changing very little for each task. For example, for text classification such as sentiment analysis, all output tokens are ignored except for the first one, corresponding to the class token, and a new output layer replaces the previous one, which was just a binary classification layer for NSP.

In February 2019, just a few months after BERT was published, Alec Radford, Jeffrey Wu, and other OpenAI researchers published the **GPT-2 paper**,³² which proposed a very similar architecture to GPT, but larger still (with over 1.5 billion parameters!). The researchers showed that the new and improved GPT model could perform *zero-shot learning* (ZSL), meaning it could achieve good performance on

many tasks without any fine-tuning. This was just the start of a race towards larger and larger models: Google’s **Switch Transformers**³³ (introduced in January 2021) used 1 trillion parameters, and soon much larger models came out, such as the Wu Dao 2.0 model by the Beijing Academy of Artificial Intelligence (BAII), in June 2021.

Sadly, this trend toward gigantic models implies that only well-funded organizations can afford to train such models: it can easily cost hundreds of thousands of dollars or more. And the energy required to train a single model corresponds to an American household’s electricity consumption for several years: not eco-friendly at all. Many of these models are just too big to even be used on regular hardware: they wouldn’t fit in RAM, and they would be horribly slow. Lastly, some are so costly that they are not released publicly.

Luckily, ingenious researchers are finding new ways to downsize Transformers and make them more data-efficient. For example, the **DistilBERT model**,³⁴ introduced in October 2019 by Victor Sanh et al. from Hugging Face, is a small and fast Transformer model based on BERT. It is available on Hugging Face’s excellent model hub, along with thousands of others—we will see an example later in this chapter. DistilBERT was trained using *distillation* (hence the name): this means transferring knowledge from a teacher model to a student one, which is usually much smaller than the teacher model. This is typically done by using the teacher’s predicted probabilities for each training instance as targets for the student. Surprisingly, distillation often works better than training the student from scratch on the same dataset as the teacher! Indeed, the student benefits from the teacher’s more nuanced labels.

Many more Transformer architectures came out after BERT almost every month, often improving on the state-of-the-art across all NLP tasks: XLNet (June 2019), RoBERTa (July 2019), StructBERT (August 2019), ALBERT (September 2019), T5 (October 2019), ELECTRA (March 2020), GPT3 (May 2020), DeBERTa (June 2020), Switch Transformers (January 2021), Wu Dao 2.0 (June 2021), Gopher (December 2021), GPT-NeoX-20B (February 2022), Chinchilla (March 2022), OPT (May 2022), and the list goes on and on. Each of these models brought new ideas and techniques,³⁵ but I particularly like Google’s **T5 paper**³⁶ by Google researchers: it framed all NLP tasks as text-to-text, using an encoder-decoder Transformer. For example, to translate “I like soccer” to Spanish, you can just call the model with the following input sentence: “translate English to Spanish: I like soccer” and it outputs “me gusta el fútbol”. To summarize a paragraph, you just enter “summarize:” followed by the paragraph, and it outputs the summary. For classification, you only need to change the prefix to “classify:” and the model outputs the class name, as text. And so on. This simplifies using the model, and it also makes it possible to pretrain it on even more tasks.

Last but not least, in April 2022, Google researchers used a new large-scale training platform named *Pathways* (which we will briefly discuss in **Chapter 19**) to train a humongous language model named **Pathways Language Model (PaLM)**³⁷, with a whopping 540 billion parameters, using over 6,000 TPUs. Other than its incredible size, this model is a standard Transformer, using decoders only (i.e., with masked multi-head attention layers), with just a few tweaks (please see the paper for the details). This model reached incredible performance in all sorts of NLP tasks, particularly in Natural Language Understanding (NLU). It’s capable of impressive feats, such as explaining jokes, giving detailed step-by-step answers to questions, or even coding. This is in part due to the model’s size, but also thanks to a technique called **Chain-of-Thought Prompting**³⁸, which was introduced a couple months earlier by another team of Google researchers.

In question answering tasks, regular prompting typically includes a few examples of questions and answers, such as: “Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now? A: 11.” The prompt then continues with the actual question, such as “Q: John takes care of 10 dogs. Each dog takes .5 hours a day to walk and take care of their business. How many hours a week does he spend taking care of dogs? A:”, and the model’s job is to append the answer: in this case, “35.” But with Chain-of-Thought Prompting, the example answers include all the reasoning steps that lead to the conclusion. For example, instead of “A: 11,” the prompt contains “A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$.” This encourages the model to give a detailed answer to the actual question, such as “John takes care of 10 dogs. Each dog takes .5 hours a day to walk and take care of their business. So that is $10 \times .5 = 5$ hours a day. 5 hours a day $\times 7$ days a week = 35 hours a week. The answer is 35 hours a week.” This is an actual example from the paper! Not only does the model give the right answer much more frequently than using regular prompting—we’re encouraging the model to think things through—it also provides all the reasoning steps, which can be useful to better understand the rationale behind a model’s answer.

Transformers have taken over NLP, but they didn’t stop there: they soon expanded to computer vision as well.

Vision Transformers

One of the first applications of attention mechanisms beyond NMT was in generating image captions using **visual attention**.³⁹ a convolutional neural network first processes the image and outputs some feature maps, then a decoder RNN equipped with an attention mechanism generates the caption, one word at a time. At each decoder time step (i.e., each word), the decoder uses the attention model to focus on just the right part of the image. For example, in **Figure 16-12**, the model generated the caption “A woman is throwing a frisbee in a park”, and you can see what part of the input image the decoder focused its attention on when it was about to output the word “frisbee”: clearly, most of its attention was focused on the frisbee.



Figure 16-12. Visual attention: an input image (left) and the model's focus before producing the word “frisbee” (right)⁴⁰

EXPLAINABILITY

One extra benefit of attention mechanisms is that they make it easier to understand what led the model to produce its output. This is called *explainability*. It can be especially useful when the model makes a mistake: for example, if an image of a dog walking in the snow is labeled as “a wolf walking in the snow”, then you can go back and check what the model focused on when it output the word “wolf”. You may find that it was paying attention not only to the dog, but also to the snow, hinting at a possible explanation: perhaps the way the model learned to distinguish dogs from wolves is by checking whether or not there’s a lot of snow around. You can then fix this by training the model with more images of wolves without snow, and dogs with snow. This example comes from a great 2016 paper⁴¹ by Marco Tulio Ribeiro et al. that uses a different approach to explainability: learning an interpretable model locally around a classifier’s prediction.

In some applications, explainability is not just a tool to debug a model; it can be a legal requirement—think of a system deciding whether or not it should grant you a loan.

When Transformers came out in 2017 and people started to experiment with them beyond NLP, they were first used alongside CNNs, without replacing them. Instead, Transformers were generally used to replace RNNs, for example, in image captioning models. Transformers became slightly more visual in a 2020 paper⁴² by Facebook researchers, which proposed a hybrid CNN-Transformer architecture for object detection. Once again, the CNN first processes the input images and outputs a set of feature maps, then these feature maps are converted to sequences and fed to a transformer, which outputs bounding box predictions. But again, most of the visual work is still done by the CNN.

Then, in October 2020, a team of Google researchers released a paper⁴³ that introduced a fully Transformer-based vision model, called a *Vision Transformer* (ViT). The idea is surprisingly simple: just chop the image into little 16×16 squares, and treat the sequence of squares as if it they were a sequence of word representations. To be more precise, the squares are first flattened into $16 \times 16 \times 3 = 768$ dimensional vectors—the 3 is for the 3 RGB color channels—then these vectors go through a linear layer that transforms them but retains their dimensionality. The resulting sequence of vectors can then be treated just like a sequence of word embeddings: this means adding positional embeddings, and passing the result to the Transformer. That’s it! This paper beat the state-of-the-art on ImageNet image classification, but to be fair they had to use over 300 million additional images for training. This makes sense since Transformers don’t have as many *inductive biases* as Convolution Neural Nets, so they need extra data just to learn things that CNNs implicitly assume.

NOTE

An inductive bias is an implicit assumption made by the model, due to its architecture. For example, linear models implicitly assume that the data is, well, linear. CNNs implicitly assume that patterns learned in one location will likely be useful in other locations as well. RNNs implicitly assume that the inputs are ordered, and that recent tokens are more important than older ones. The more inductive biases a model has, assuming they are correct, the less training data the model will require. But if the implicit assumptions are wrong, then the model may perform poorly even if it is trained on a large dataset.

Just two months later, a team of Facebook researchers released a paper⁴⁴ that introduced *Data-efficient Image Transformers* (DeiT). Their model reached competitive results on ImageNet without requiring any additional data for training. The model’s architecture is virtually the same as the original ViT, but they used a distillation technique to transfer knowledge from state-of-the-art CNN models to their model.

Then in March 2021, DeepMind released an important paper⁴⁵ which introduced the *Perceiver* architecture. It is a *multimodal* Transformer, meaning you can feed it text, images, audio, or virtually any other *modality*. Until then, Transformers were restricted to fairly short sequences, because of the performance and RAM bottleneck in the attention layers. This excluded modalities such as audio or video, and it forced researchers to treat images as sequences of patches, rather than sequences of pixels. The bottleneck is due to self-attention, where every token must attend to every other token. Indeed, if the input sequence has M tokens, then the attention layer must compute an $M \times M$ matrix, which can be huge if M is very large. The Perceiver solves this problem by gradually improving a fairly short *latent representation* of the inputs, composed of N tokens—typically just a few hundred. The word *latent* means hidden, or internal. The model uses cross-attention layers only, feeding them the latent representation as the queries, and the (possibly large) inputs as the values. This only requires computing an $M \times N$ matrix, so the computational complexity is linear with regards to M , instead of quadratic. After going through several cross-attention layers, if everything goes well, the latent

representations end up capturing everything that matters in the inputs. The authors also suggested sharing the weights between consecutive cross-attention layers: if you do that, then the Perceiver effectively becomes an RNN. Indeed, the shared cross-attention layers can be seen as the same memory cell at different time steps, and the latent representation corresponds to the cell's context vector. The same inputs are repeatedly fed to the memory cell at every time step. It looks like RNNs are not dead after all!

Just a month later, Mathilde Caron et al. introduced **DINO** “Emerging Properties in Self-Supervised Vision Transformers” (2021), an impressive vision Transformer trained entirely without labels, using self-supervision, and capable of high-accuracy semantic segmentation. The model is duplicated during training, with one network acting as a teacher, and the other acting as a student. Gradient descent only affects the student, while the teacher's weights are just an exponential moving average of the student's weights. The student is trained to match the teacher's predictions: since they're almost the same model, this is called *self-distillation*. At each training step, the input images are augmented in different ways for the teacher and the student, so they don't see the exact same image, but their predictions must match. This forces them to come up with high-level representations. To prevent *mode collapse*, where both the student and the teacher would always output the same thing, completely ignoring the inputs, DINO keeps track of a moving average of the teacher's outputs, and it tweaks the teacher's predictions to ensure that they remained centered on zero, on average. DINO also forces the teacher to have high confidence in its predictions: this is called *sharpening*. Together, these techniques preserve diversity in the teacher's outputs.

In a **2021 paper**,⁴⁶ Google researchers showed how to scale ViTs up or down, depending on the amount of data. They managed to create a huge 2 billion parameter model that reached over 90.4% top-1 accuracy on ImageNet. Conversely, they also trained a scaled down model that reached over 84.8% top-1 accuracy on ImageNet, using only 10,000 images: that's just 10 images per class!

And progress in visual Transformers has continued steadily to this day. For example, in March 2022, a **paper**⁴⁷ by Mitchell Wortsman et al. demonstrated that it's possible to first train multiple Transformers, then average their weights to create a new and improved model. This is similar to an ensemble, except there's just one model in the end, which also means there's no inference time penalty.

Now the latest trend in Transformers consists in building large multimodal models, often capable of zero-shot or few-shot learning. For example, **OpenAI's 2021 CLIP paper**⁴⁸ proposed a large Transformer model pretrained to match captions with images: this task allows it to learn excellent image representations, and the model can then be used directly for tasks such as image classification using simple text prompts such as “a photo of a cat”. Soon after, OpenAI released the **DALL·E paper**⁴⁹, capable of generating amazing images based on text prompts. The **DALL·E 2 paper**⁵⁰ generates even higher quality images using a diffusion model (see **Chapter 17**). In April 2022, DeepMind released the **Flamingo paper**⁵¹, which introduced a family of models pretrained on a wide variety of tasks across multiple modalities, including text, images, and videos. A single model can be used across very different tasks, such as question-answering, image captioning, and more. Soon after, in May 2022, DeepMind introduced the **GATO model**⁵², again a multimodal model, but which can be used as a policy for a Reinforcement Learning agent (RL will be introduced in **Chapter 18**). The same Transformer can chat with you, caption images, play Atari games, control (simulated) robotic arms, and more. And all that with “only” 1.2 billion parameters. And the adventure continues!

NOTE

These astounding advances have led some researchers to claim that Human-Level AI is near, that “scale is all you need,” and that some of these models may be “slightly conscious.” Others point out that despite the amazing progress, these models still lack the reliability and adaptability of human intelligence, our ability to reason symbolically, to generalize based on a single example, and more.

As you can see, Transformers are everywhere! And the good news is that you generally won’t have to implement Transformers yourself since many excellent pretrained models are readily available for download via TensorFlow Hub or Hugging Face’s model hub. We have already seen how to use a model from TF Hub, so let’s close this chapter by taking a quick look at Hugging Face’s ecosystem.

Hugging Face’s Transformers Library

It’s impossible to talk about Transformers today without mentioning Hugging Face, an A.I. company that has built a whole ecosystem of easy-to-use open source tools for NLP, vision, and beyond. The central component of their ecosystem is the *Transformers library*, which allows you to easily download a pretrained model, including its corresponding tokenizer, and then fine-tune it on your own dataset, if needed. Plus the library supports TensorFlow, PyTorch, and JAX (with the Flax library).

The simplest way to use the Transformers library is to use the `transformers.pipeline()` function: you just specify which task you want, such as sentiment analysis, and it downloads a default pretrained model, ready to be used—it really couldn’t be any simpler:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis") # many other tasks are available
result = classifier("The actors were very convincing.")
```

The result is a Python list containing one dictionary per input text:

```
>>> result
[{'label': 'POSITIVE', 'score': 0.9998071789741516}]
```

In this example, the model correctly found that the sentence is positive, with around 99.98% confidence. Of course, you can also pass a batch of sentences to the model:

```
>>> classifier(["I am from India.", "I am from Iraq."])
[{'label': 'POSITIVE', 'score': 0.9896161556243896},
 {'label': 'NEGATIVE', 'score': 0.9811071157455444}]
```


BIAS AND FAIRNESS

As the output suggests, this specific classifier loves Indians, but is severely biased against Iraqis. You can try this code with your own country or city. Such an undesirable bias generally comes in large part from the training data itself: in this case, there were plenty of negative sentences related to the wars in Iraq in the training data. This bias was then amplified during the fine-tuning process since the model was forced to choose between just two classes: positive or negative. If you add a neutral class when fine-tuning, then the country bias mostly disappears. But the training data is not the only source of bias: the model's architecture, the type of loss or regularization used for training, the optimizer, all of these can affect what the model ends up learning. Even a mostly unbiased model can be *used* in a biased way, much like survey questions can be biased.

Understanding bias in A.I. and mitigating its negative effects is still an area of active research, but one thing is certain: you should pause and think before you rush to deploy a model to production. Ask yourself how the model could do harm, even indirectly. For example, if the model's predictions are used to decide whether or not to give someone a loan, the process should be fair. So make sure you evaluate the model's performance not just on average over the whole test set, but across various subsets as well: for example, you may find that although the model works very well on average, its performance is abysmal for some categories of people. You may also want to run counterfactual tests: for example, you may want to check that the model's predictions do not change when you simply switch someone's gender.

If the model works well on average, it's tempting to push it to production and move on to something else, especially if it's just one component of a much larger system. But in general, if you don't fix such issues, no one else will, and your model may end up doing more harm than good. The solution depends on the problem: it may require rebalancing the dataset, fine-tuning on a different dataset, switching to another pretrained model, tweaking the model's architecture or hyperparameters, etc.

The `pipeline()` function uses the default model for the given task. For example, for text classification such as sentiment analysis, at the time of writing, it defaults to “distilbert-base-uncased-finetuned-sst-2-english”—a DistilBERT model with an uncased tokenizer, trained on the English Wikipedia and on a corpus of English books, and fine-tuned on the Stanford Sentiment Treebank v2 (SST 2) task. It's also possible to manually specify a different model. For example, you could use a DistilBERT model fine-tuned on the Multi-Genre Natural Language Inference (MNLI) task, which classifies two sentences into three classes: contradiction, neutral, or entailment. Here is how:

```
>>> model_name = "huggingface/distilbert-base-uncased-finetuned-mnli"
>>> classifier_mnli = pipeline("text-classification", model=model_name)
>>> classifier_mnli("She loves me. [SEP] She loves me not.")
[{'label': 'contradiction', 'score': 0.9790192246437073}]
```

TIP

You can find the available models at <https://huggingface.co/models>, and the list of tasks at <https://huggingface.co/tasks>.

The pipeline API is very simple and convenient, but sometimes you will need more control. For such cases, the Transformers library provides many classes, including all sorts of tokenizers, models, configurations, callbacks, and much more. For example, let's load the same DistilBERT model, along with its corresponding tokenizer, using the `TFAutoModelForSequenceClassification` and `AutoTokenizer` classes:

```
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = TFAutoModelForSequenceClassification.from_pretrained(model_name)
```

Next, let's tokenize a couple pairs of sentences. In this code, we activate padding and specify that we want TensorFlow tensors instead of Python lists:

```
token_ids = tokenizer(["I like soccer. [SEP] We all love soccer!",
                      "Joe lived for a very long time. [SEP] Joe is old."],
                     padding=True, return_tensors="tf")
```

TIP

Instead of passing "Sentence 1 [SEP] Sentence 2" to the tokenizer, you can equivalently pass it a tuple: ("Sentence 1", "Sentence 2").

The output is a dictionary-like instance of the `BatchEncoding` class, which contains the sequences of token IDs, as well as a mask containing 0s for the padding tokens:

```
>>> token_ids
{'input_ids': <tf.Tensor: shape=(2, 15), dtype=int32, numpy=
array([[ 101, 1045, 2066, 4715, 1012,  102, 2057, 2035, 2293, 4715,  999,
        102,    0,    0,    0],
       [ 101, 3533, 2973, 2005, 1037, 2200, 2146, 2051, 1012,  102, 3533,
        2003, 2214, 1012,  102]]), dtype=int32)>,
 'attention_mask': <tf.Tensor: shape=(2, 15), dtype=int32, numpy=
array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0],
       [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]]), dtype=int32)>}
```

If you set `return_token_type_ids=True` when calling the tokenizer, you will also get an extra tensor that indicates which sentence each token belongs to. This is needed by some models, but not DistilBERT.

Next, we can directly pass this `BatchEncoding` object to the model: it returns a `TFSequenceClassifierOutput` object containing its predicted class logits:

```
>>> outputs = model(token_ids)
>>> outputs
TFSequenceClassifierOutput(loss=None, logits=[<tf.Tensor: [...] numpy=
array([[-2.1123817 ,  1.1786783 ,  1.4101017 ],
       [-0.01478387,  1.0962474 , -0.9919954 ]], dtype=float32)>], [...])
```

Lastly, we can apply the softmax activation function to convert these logits to class probabilities, and use the `argmax()` function to predict the class with the highest probability, for each input sentence

pair:

```
>>> Y_probas = tf.keras.activations.softmax(outputs.logits)
>>> Y_probas
<tf.Tensor: shape=(2, 3), dtype=float32, numpy=
array([[0.01619702, 0.43523544, 0.5485676 ],
       [0.08672056, 0.85204804, 0.06123142]], dtype=float32)>
>>> Y_pred = tf.argmax(Y_probas, axis=1)
>>> Y_pred # 0 = contradiction, 1 = entailment, 2 = neutral
<tf.Tensor: shape=(2,), dtype=int64, numpy=array([2, 1])>
```

In this example, the model correctly classifies the first sentence pair as neutral—the fact that I like soccer does not imply that everyone else does—and the second pair as an entailment—Joe must indeed be quite old.

If you wish to fine-tune this model on your own dataset, you can train the model as usual with Keras since it's just a regular Keras model with a few extra methods. However, because the model outputs logits instead of probabilities, you must use the `tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)` loss instead of the usual `"sparse_categorical_crossentropy"` loss. Moreover, the model does not support `BatchEncoding` inputs during training, so we must use its `data` attribute to get a regular dictionary instead.

```
sentences = [("Sky is blue", "Sky is red"), ("I love her", "She loves me")]
X_train = tokenizer(sentences, padding=True, return_tensors="tf").data
y_train = tf.constant([0, 2]) # contradiction, neutral
loss = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
model.compile(loss=loss, optimizer="nadam", metrics=["accuracy"])
history = model.fit(X_train, y_train, epochs=2)
```

Hugging Face has also built a *Datasets library* that you can use to easily download a standard dataset (such as IMDb) or a custom one, and use it to fine-tune your model. It's similar to TensorFlow Datasets, but it also provides tools to perform common preprocessing tasks on the fly, such as masking. The list of datasets is available at <https://huggingface.co/datasets>.

This should get you started with Hugging Face's ecosystem. To learn more, you can head over to <https://huggingface.co/docs> for the documentation, which includes many tutorial notebooks, videos, the full API, and more. I also recommend you check out the O'Reilly book *Natural Language Processing with Transformers: Building Language Applications with Hugging Face* by Lewis Tunstall, Leandro von Werra, and Thomas Wolf—all from the Hugging Face team.

In the next chapter we will discuss how to learn deep representations in an unsupervised way using autoencoders, and we will use generative adversarial networks (GANs) to produce images and more!

Exercises

1. What are the pros and cons of using a stateful RNN versus a stateless RNN?
2. Why do people use encoder–decoder RNNs rather than plain sequence-to-sequence RNNs for automatic translation?

3. How can you deal with variable-length input sequences? What about variable-length output sequences?
4. What is beam search and why would you use it? What tool can you use to implement it?
5. What is an attention mechanism? How does it help?
6. What is the most important layer in the Transformer architecture? What is its purpose?
7. When would you need to use sampled softmax?
8. *Embedded Reber grammars* were used by Hochreiter and Schmidhuber in [their paper](#) about LSTMs. They are artificial grammars that produce strings such as “BPBTSXXVPSEPE.” Check out Jenny Orr’s [nice introduction](#) to this topic. Choose a particular embedded Reber grammar (such as the one represented on Jenny Orr’s page), then train an RNN to identify whether a string respects that grammar or not. You will first need to write a function capable of generating a training batch containing about 50% strings that respect the grammar, and 50% that don’t.
9. Train an encoder–decoder model that can convert a date string from one format to another (e.g., from “April 22, 2019” to “2019-04-22”).
10. Go through the example on Keras’ site for [Natural language image search with a Dual Encoder](#). You will learn how to build a model capable of representing both images and text within the same embedding space. This makes it possible to search for images using a text prompt, like in the CLIP model by OpenAI.
11. Use the Hugging Face Transformers library to download a pretrained language model capable of generating text (e.g., GPT), and try generating more convincing Shakespearean text. You will need to use the model’s `generate()` method—see Hugging Face’s documentation for more details.

Solutions to these exercises are available at the end of this chapter’s notebook, at <https://homl.info/colab3>.

-
- 1 Alan Turing, “Computing Machinery and Intelligence,” *Mind* 49 (1950): 433–460.
 - 2 Of course, the word *chatbot* came much later. Turing called his test the *imitation game*: machine A and human B chat with human interrogator C via text messages; the interrogator asks questions to figure out which one is the machine (A or B). The machine passes the test if it can fool the interrogator, while the human B must try to help the interrogator.
 - 3 In this chapter, when discussing input/output strings, I chose to violate the American punctuation rule that requires dots and commas to be inside the quotes, to avoid any confusion. My apologies to the purists.
 - 4 Since the input windows overlap, the concept of epoch is not so clear in this case: during each epoch (as implemented by Keras), the model will actually see the same character multiple times.
 - 5 Alec Radford et al., “Learning to Generate Reviews and Discovering Sentiment,” arXiv preprint arXiv:1704.01444 (2017).
 - 6 Rico Sennrich et al., “Neural Machine Translation of Rare Words with Subword Units,” *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics* 1 (2016): 1715–1725.
 - 7 Taku Kudo, “Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates,” arXiv preprint arXiv:1804.10959 (2018).
 - 8 Taku Kudo and John Richardson, “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing,” arXiv preprint arXiv:1808.06226 (2018).
 - 9 Yonghui Wu et al., “Google’s Neural Machine Translation System: Bridging the Gap Between Human and Machine ;Translation,” arXiv preprint arXiv:1609.08144 (2016).

- 10 Ragged tensors were introduced in [Chapter 12](#), and they are detailed in [Appendix C](#).
- 11 Matthew Peters et al., “Deep Contextualized Word Representations,” *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies 1* (2018): 2227–2237.
- 12 Jeremy Howard and Sebastian Ruder, “Universal Language Model Fine-Tuning for Text Classification,” *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics 1* (2018): 328–339.
- 13 Daniel Cer et al., “Universal Sentence Encoder,” arXiv preprint arXiv:1803.11175 (2018)
- 14 Ilya Sutskever et al., “Sequence to Sequence Learning with Neural Networks,” arXiv:1409.3215 (2014).
- 15 Samy Bengio et al., “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks,” arXiv preprint arXiv:1506.03099 (2015).
- 16 This dataset is composed of sentence pairs created by contributors of the [Tatoeba Project](#). About 120,000 sentence pairs were selected by the authors of the website <https://manythings.org/anki>. This dataset is released under the Creative Commons Attribution 2.0 France license. Other language pairs are available.
- 17 In Python, if you run `a, *b = [1, 2, 3, 4]`, then `a` equals 1 and `b` equals `[2, 3, 4]`.
- 18 Sébastien Jean et al., “On Using Very Large Target Vocabulary for Neural Machine Translation,” *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing 1* (2015): 1–10.
- 19 Dzmitry Bahdanau et al., “Neural Machine Translation by Jointly Learning to Align and Translate,” arXiv preprint arXiv:1409.0473 (2014).
- 20 Minh-Thang Luong et al., “Effective Approaches to Attention-Based Neural Machine Translation,” *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing* (2015): 1412–1421.
- 21 Ashish Vaswani et al., “Attention Is All You Need,” *Proceedings of the 31st International Conference on Neural Information Processing Systems* (2017): 6000–6010.
- 22 Since the Transformer uses time-distributed dense layers, you could argue that it uses 1D convolutional layers with a kernel size of 1.
- 23 This is figure 1 from the “Attention Is All You Need” paper, reproduced with the kind permission of the authors.
- 24 It’s possible to use ragged tensors instead, if you are using the latest version of TensorFlow.
- 25 This is the right part of figure 2 from the “Attention is all you need” paper, reproduced with the kind authorization of the authors.
- 26 This will most likely change by the time you read this. Check out Keras issue #16248 for more details. When this happens, there will be no need to set the `attention_mask` argument, and therefore no need to create `encoder_pad_mask`.
- 27 Currently `Z + skip` does not support automatic masking, which is why we had to write `tf.keras.layers.Add()([Z, skip])` instead. Again, this may change by the time you read this.
- 28 Alec Radford et al., “Improving Language Understanding by Generative Pre-Training” (2018).
- 29 For example, the sentence “Jane had a lot of fun at her friend’s birthday party” entails “Jane enjoyed the party,” but it is contradicted by “Everyone hated the party” and it is unrelated to “The Earth is flat.”
- 30 Jacob Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies 1* (2019).
- 31 This is figure 1 from the paper, reproduced with the kind authorization of the authors.
- 32 Alec Radford et al., “Language Models Are Unsupervised Multitask Learners” (2019).
- 33 William Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity” (2021).
- 34 Victor Sanh et al., “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter” (2019).
- 35 Mariya Yao summarized many of these models in this post: <https://hml.info/yaopost>.
- 36 Colin Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer” (2019).
- 37 Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, et al., “PaLM: Scaling Language Modeling with Pathways” (2022).
- 38 Jason Wei et al., “Chain of Thought Prompting Elicits Reasoning in Large Language Models” (2022)
- 39 Kelvin Xu et al., “Show, Attend and Tell: Neural Image Caption Generation with Visual Attention,” *Proceedings of the 32nd International Conference on Machine Learning* (2015): 2048–2057.

- 40 This is a part of figure 3 from the paper. It is reproduced with the kind authorization of the authors.
- 41 Marco Tulio Ribeiro et al., “‘Why Should I Trust You?’: Explaining the Predictions of Any Classifier,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2016): 1135–1144.
- 42 Nicolas Carion et al., “End-to-End Object Detection with Transformers” (2020).
- 43 Alexey Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale” (2020).
- 44 Hugo Touvron et al., “Training data-efficient image transformers & distillation through attention” (2020).
- 45 Andrew Jaegle et al., “Perceiver: General Perception with Iterative Attention” (2021).
- 46 Xiaohua Zhai et al., “Scaling Vision Transformers” (2021).
- 47 Mitchell Wortsman et al., “Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time” (2022).
- 48 Alec Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” (2021).
- 49 Aditya Ramesh et al., “Zero-Shot Text-to-Image Generation,” (2021).
- 50 “Hierarchical Text-Conditional Image Generation with CLIP Latents,” (2022).
- 51 Jean-Baptiste Alayrac et al., “Flamingo: a Visual Language Model for Few-Shot Learning,” (2022).
- 52 Scott Reed et al., “A Generalist Agent” (2022).

Chapter 17. Autoencoders, GANs, and Diffusion Models

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 17th chapter of the final book. The GitHub repo is <https://github.com/ageron/handson-ml3>.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at mcronin@oreilly.com.

Autoencoders are artificial neural networks capable of learning dense representations of the input data, called *latent representations* or *codings*, without any supervision (i.e., the training set is unlabeled). These codings typically have a much lower dimensionality than the input data, making autoencoders useful for dimensionality reduction (see [Chapter 8](#)), especially for visualization purposes. Autoencoders also act as feature detectors, and they can be used for unsupervised pretraining of deep neural networks (as we discussed in [Chapter 11](#)). Lastly, some autoencoders are *generative models*: they are capable of randomly generating new data that looks very similar to the training data. For example, you could train an autoencoder on pictures of faces, and it would then be able to generate new faces.

Generative adversarial networks (GANs) are also neural nets capable of generating data. In fact, they can generate pictures of faces so convincing that it is hard to believe the people they represent do not exist. You can judge so for yourself by visiting <https://thispersondoesnotexist.com/>, a website that shows faces generated by a GAN architecture called *StyleGAN*.

You can also check out <https://thisrentaldoesnotexist.com/> to see some generated Airbnb listings. GANs are now widely used for super resolution (increasing the resolution of an image), **colorization**, powerful image editing (e.g., replacing photo bombers with realistic background), turning a simple sketch into a photorealistic image, predicting the next frames in a video, augmenting a dataset (to train other models), generating other types of data (such as text, audio, and time series), identifying the weaknesses in other models to strengthen them, and more.

Lastly, *diffusion models* have also joined the generative learning party. In 2021, they managed to generate more diverse and higher quality images than GANs, while also being much easier to train. However, diffusion models are much slower to run.

Autoencoders, GANs, and diffusion models are all unsupervised, they all learn latent representations, they can all be used as generative models, and they have many similar applications. However, they work very differently:

- Autoencoders simply learn to copy their inputs to their outputs. This may sound like a trivial task, but we will see that constraining the network in various ways can make it rather difficult. For example, you can limit the size of the latent representations, or you can add noise to the inputs and train the network to recover the original inputs. These constraints prevent the autoencoder from trivially copying the inputs directly to the outputs, which forces it to learn efficient ways of representing the data. In short, the codings are byproducts of the autoencoder learning the identity function under some constraints.
- GANs are composed of two neural networks: a *generator* that tries to generate data that looks similar to the training data, and a *discriminator* that tries to tell real data from fake data. This architecture is very original in Deep Learning in that the generator and the discriminator compete against each other during training: the generator is often compared to a criminal trying to make realistic counterfeit money, while the discriminator is like the police investigator trying to tell real money from fake. *Adversarial training*

(training competing neural networks) is widely considered as one of the most important ideas of the 2010s. In 2016, Yann LeCun even said that it was “the most interesting idea in the last 10 years in Machine Learning.”

- A *Denoising Diffusion Probabilistic Model* (DDPM) is trained to remove a tiny bit of noise from an image. If you then take an image entirely full of Gaussian noise, and repeatedly run the diffusion model on that image, a high quality image will gradually emerge, similar to the training images (but not identical).

In this chapter we will start by exploring in more depth how autoencoders work and how to use them for dimensionality reduction, feature extraction, unsupervised pretraining, or as generative models. This will naturally lead us to GANs. We will start by building a simple GAN to generate fake images, but we will see that training is often quite difficult. We will discuss the main difficulties you will encounter with adversarial training, as well as some of the main techniques to work around these difficulties. And lastly we will build and train a DDPM and use it to generate images. Let’s start with autoencoders!

Efficient Data Representations

Which of the following number sequences do you find the easiest to memorize?

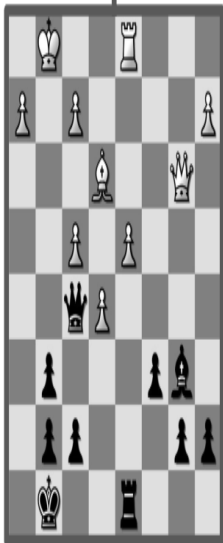
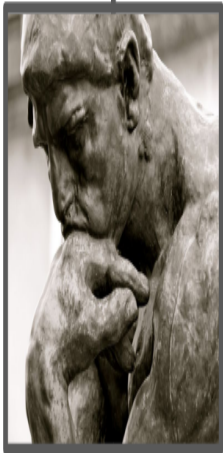
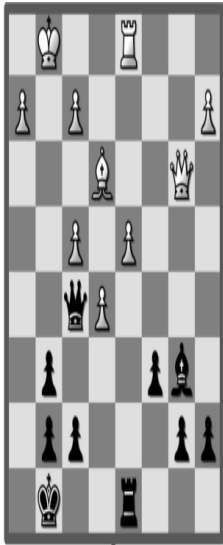
- 40, 27, 25, 36, 81, 57, 10, 73, 19, 68
- 50, 48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18, 16, 14

At first glance, it would seem that the first sequence should be easier, since it is much shorter. However, if you look carefully at the second sequence, you will notice that it is just the list of even numbers from 50 down to 14. Once you notice this pattern, the second sequence becomes much easier to memorize than the first because you only need to remember the pattern (i.e., decreasing even numbers) and the starting and ending numbers (i.e., 50 and

14). Note that if you could quickly and easily memorize very long sequences, you would not care much about the existence of a pattern in the second sequence. You would just learn every number by heart, and that would be that. The fact that it is hard to memorize long sequences is what makes it useful to recognize patterns, and hopefully this clarifies why constraining an autoencoder during training pushes it to discover and exploit patterns in the data.

The relationship between memory, perception, and pattern matching was famously studied by William Chase and Herbert Simon in the early 1970s.¹ They observed that expert chess players were able to memorize the positions of all the pieces in a game by looking at the board for just five seconds, a task that most people would find impossible. However, this was only the case when the pieces were placed in realistic positions (from actual games), not when the pieces were placed randomly. Chess experts don't have a much better memory than you and I; they just see chess patterns more easily, thanks to their experience with the game. Noticing patterns helps them store information efficiently.

Just like the chess players in this memory experiment, an autoencoder looks at the inputs, converts them to an efficient latent representation, and then spits out something that (hopefully) looks very close to the inputs. An autoencoder is always composed of two parts: an *encoder* (or *recognition network*) that converts the inputs to a latent representation, followed by a *decoder* (or *generative network*) that converts the internal representation to the outputs (see Figure 17-1).



Outputs
($\approx$ inputs)

Latent
representation

Inputs

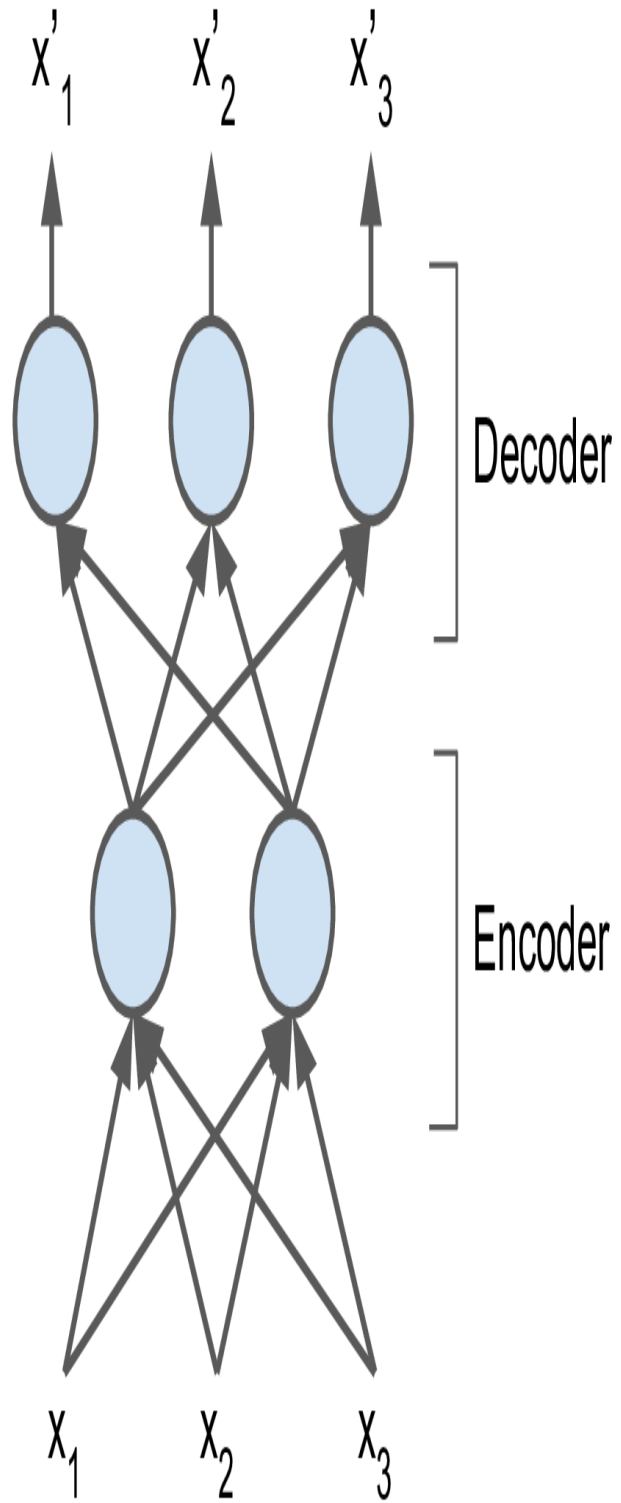


Figure 17-1. The chess memory experiment (left) and a simple autoencoder (right)

As you can see, an autoencoder typically has the same architecture as a Multi-Layer Perceptron (MLP; see [Chapter 10](#)), except that the number of neurons in the output layer must be equal to the number of inputs. In this example, there is just one hidden layer composed of two neurons (the encoder), and one output layer composed of three neurons (the decoder). The outputs are often called the *reconstructions* because the autoencoder tries to reconstruct the inputs. The cost function contains a *reconstruction loss* that penalizes the model when the reconstructions are different from the inputs.

Because the internal representation has a lower dimensionality than the input data (it is 2D instead of 3D), the autoencoder is said to be *undercomplete*. An undercomplete autoencoder cannot trivially copy its inputs to the codings, yet it must find a way to output a copy of its inputs. It is forced to learn the most important features in the input data (and drop the unimportant ones).

Let's see how to implement a very simple undercomplete autoencoder for dimensionality reduction.

Performing PCA with an Undercomplete Linear Autoencoder

If the autoencoder uses only linear activations and the cost function is the mean squared error (MSE), then it ends up performing Principal Component Analysis (PCA; see [Chapter 8](#)).

The following code builds a simple linear autoencoder to perform PCA on a 3D dataset, projecting it to 2D:

```
import tensorflow as tf

encoder = tf.keras.Sequential([tf.keras.layers.Dense(2)])
decoder = tf.keras.Sequential([tf.keras.layers.Dense(3)])
autoencoder = tf.keras.Sequential([encoder, decoder])
```

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.5)
autoencoder.compile(loss="mse", optimizer=optimizer)
```

This code is really not very different from all the MLPs we built in past chapters, but there are a few things to note:

- We organized the autoencoder into two subcomponents: the encoder and the decoder. Both are regular `Sequential` models with a single `Dense` layer each, and the autoencoder is a `Sequential` model containing the encoder followed by the decoder (remember that a model can be used as a layer in another model).
- The autoencoder's number of outputs is equal to the number of inputs (i.e., 3).
- To perform PCA, we do not use any activation function (i.e., all neurons are linear), and the cost function is the MSE. That's because PCA is a linear transformation. We will see more complex and non-linear autoencoders shortly.

Now let's train the model on the same simple generated 3D dataset we used in [Chapter 8](#) and use it to encode that dataset (i.e., project it to 2D):

```
X_train = [...] # generate a 3D dataset, like in Chapter 8
history = autoencoder.fit(X_train, X_train, epochs=500,
                           verbose=False)
codings = encoder.predict(X_train)
```

Note that `X_train` is used as both the inputs and the targets. [Figure 17-2](#) shows the original 3D dataset (on the left) and the output of the autoencoder's hidden layer (i.e., the coding layer, on the right). As you can see, the autoencoder found the best 2D plane to project the data onto, preserving as much variance in the data as it could (just like PCA).

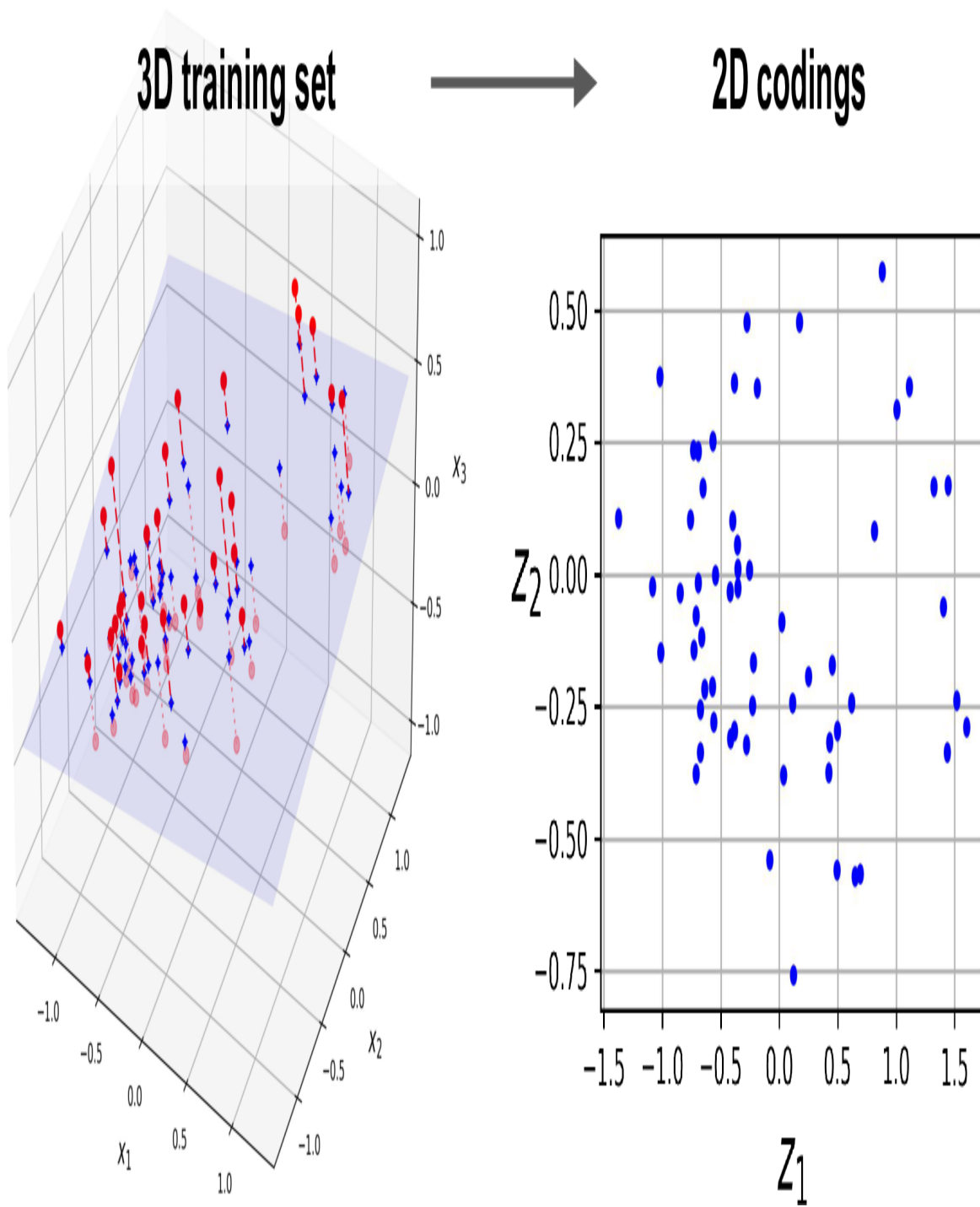


Figure 17-2. Approximate PCA performed by an undercomplete linear autoencoder

NOTE

You can think of autoencoders as a form of self-supervised learning since it is based on a supervised learning technique with automatically generated labels, in this case simply equal to the inputs.

Stacked Autoencoders

Just like other neural networks we have discussed, autoencoders can have multiple hidden layers. In this case they are called *stacked autoencoders* (or *deep autoencoders*). Adding more layers helps the autoencoder learn more complex codings. That said, one must be careful not to make the autoencoder too powerful. Imagine an encoder so powerful that it just learns to map each input to a single arbitrary number (and the decoder learns the reverse mapping). Obviously such an autoencoder will reconstruct the training data perfectly, but it will not have learned any useful data representation in the process, and it is unlikely to generalize well to new instances.

The architecture of a stacked autoencoder is typically symmetrical with regard to the central hidden layer (the coding layer). To put it simply, it looks like a sandwich. For example, an autoencoder for Fashion MNIST (introduced in [Chapter 10](#)) may have 784 inputs, followed by a hidden layer with 100 neurons, then a central hidden layer of 30 neurons, then another hidden layer with 100 neurons, and an output layer with 784 neurons. This stacked autoencoder is represented in [Figure 17-3](#).

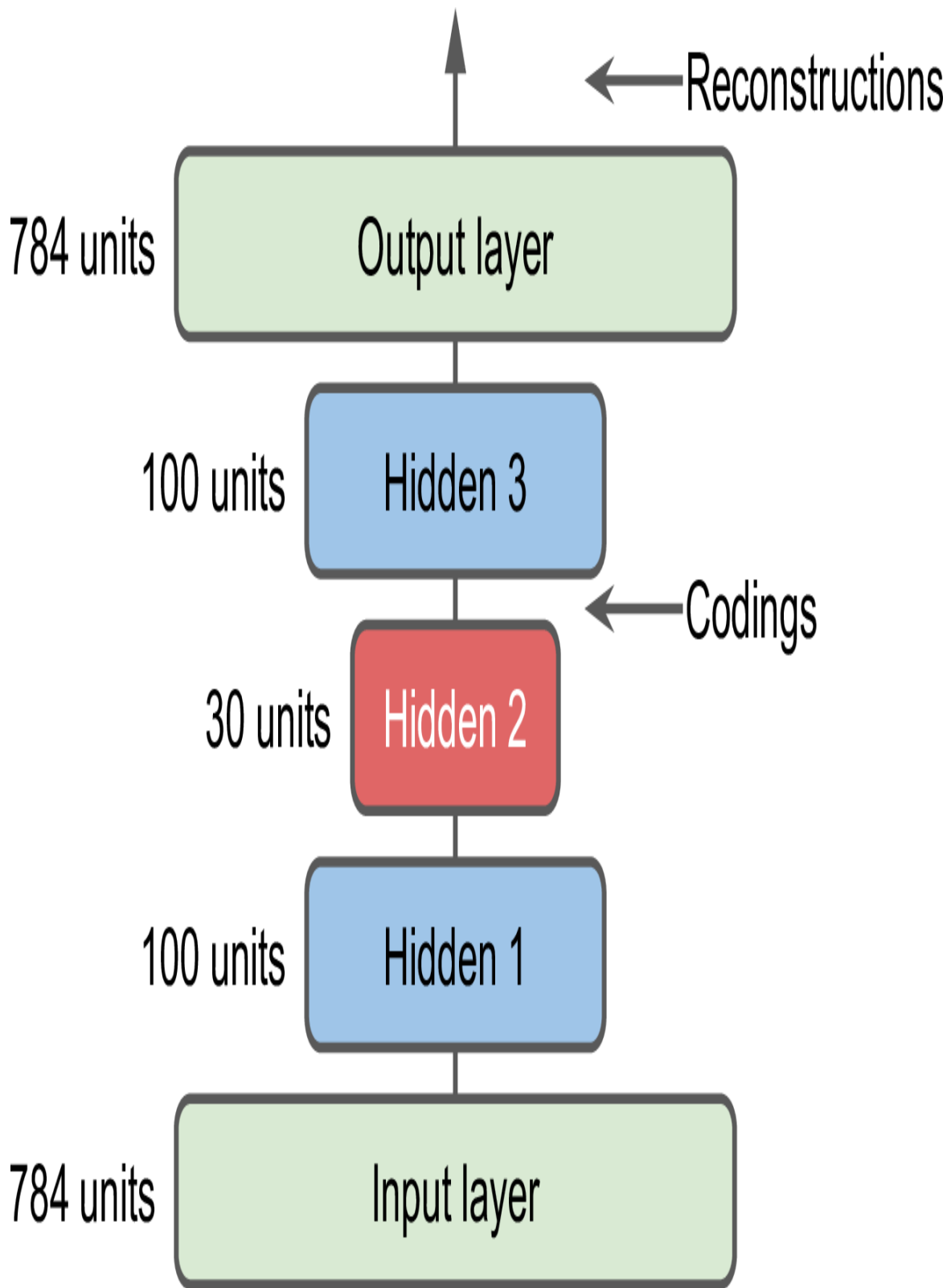


Figure 17-3. Stacked autoencoder

Implementing a Stacked Autoencoder Using Keras

You can implement a stacked autoencoder very much like a regular deep MLP:

```
stacked_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu"),
])
stacked_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
stacked_ae = tf.keras.Sequential([stacked_encoder,
stacked_decoder])

stacked_ae.compile(loss="mse", optimizer="nadam")
history = stacked_ae.fit(X_train, X_train, epochs=20,
                        validation_data=(X_valid, X_valid))
```

Let's go through this code:

- Just like earlier, we split the autoencoder model into two submodels: the encoder and the decoder.
- The encoder takes 28×28 -pixel grayscale images, flattens them so that each image is represented as a vector of size 784, then processes these vectors through two Dense layers of diminishing sizes (100 units then 30 units), both using the ReLU activation function. For each input image, the encoder outputs a vector of size 30.
- The decoder takes codings of size 30 (output by the encoder) and processes them through two Dense layers of increasing sizes (100 units then 784 units), and it reshapes the final vectors into 28×28 arrays so the decoder's outputs have the same shape as the encoder's inputs.
- When compiling the stacked autoencoder, we use the MSE loss, and Nadam optimization.

- Finally, we train the model using `X_train` as both the inputs and the targets. Similarly, we use `X_valid` as both the validation inputs and targets.

Visualizing the Reconstructions

One way to ensure that an autoencoder is properly trained is to compare the inputs and the outputs: the differences should not be too significant. Let's plot a few images from the validation set, as well as their reconstructions:

```
import numpy as np

def plot_reconstructions(model, images=X_valid, n_images=5):
    reconstructions = np.clip(model.predict(images[:n_images]),
0, 1)
    fig = plt.figure(figsize=(n_images * 1.5, 3))
    for image_index in range(n_images):
        plt.subplot(2, n_images, 1 + image_index)
        plt.imshow(images[image_index], cmap="binary")
        plt.axis("off")
        plt.subplot(2, n_images, 1 + n_images + image_index)
        plt.imshow(reconstructions[image_index], cmap="binary")
        plt.axis("off")

plot_reconstructions(stacked_ae)
plt.show()
```

Figure 17-4 shows the resulting images.

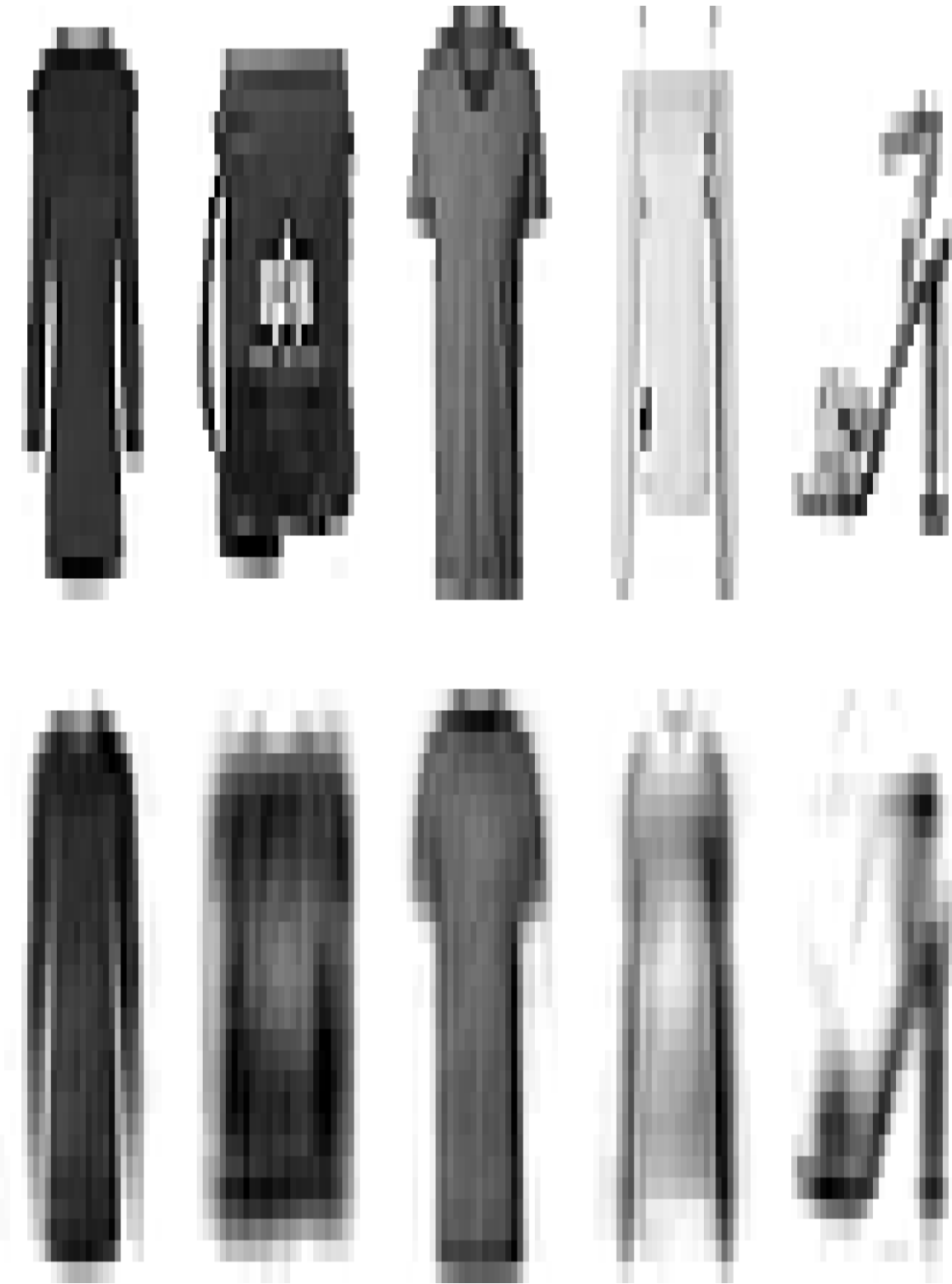


Figure 17-4. Original images (top) and their reconstructions (bottom)

The reconstructions are recognizable, but a bit too lossy. We may need to train the model for longer, or make the encoder and decoder deeper, or make the codings larger. But if we make the network too powerful, it will manage to make perfect reconstructions without having learned any useful patterns in the data. For now, let's go with this model.

Visualizing the Fashion MNIST Dataset

Now that we have trained a stacked autoencoder, we can use it to reduce the dataset's dimensionality. For visualization, this does not give great results compared to other dimensionality reduction algorithms (such as those we discussed in [Chapter 8](#)), but one big advantage of autoencoders is that they can handle large datasets, with many instances and many features. So one strategy is to use an autoencoder to reduce the dimensionality down to a reasonable level, then use another dimensionality reduction algorithm for visualization. Let's use this strategy to visualize Fashion MNIST. First, we use the encoder from our stacked autoencoder to reduce the dimensionality down to 30, then we use Scikit-Learn's implementation of the t-SNE algorithm to reduce the dimensionality down to 2 for visualization:

```
from sklearn.manifold import TSNE

X_valid_compressed = stacked_encoder.predict(X_valid)
tsne = TSNE(init="pca", learning_rate="auto", random_state=42)
X_valid_2D = tsne.fit_transform(X_valid_compressed)
```

Now we can plot the dataset:

```
plt.scatter(X_valid_2D[:, 0], X_valid_2D[:, 1], c=y_valid, s=10,
            cmap="tab10")
plt.show()
```

[Figure 17-5](#) shows the resulting scatterplot, beautified a bit by displaying some of the images. The t-SNE algorithm identified several clusters which match the classes reasonably well (each class is represented by a different color).

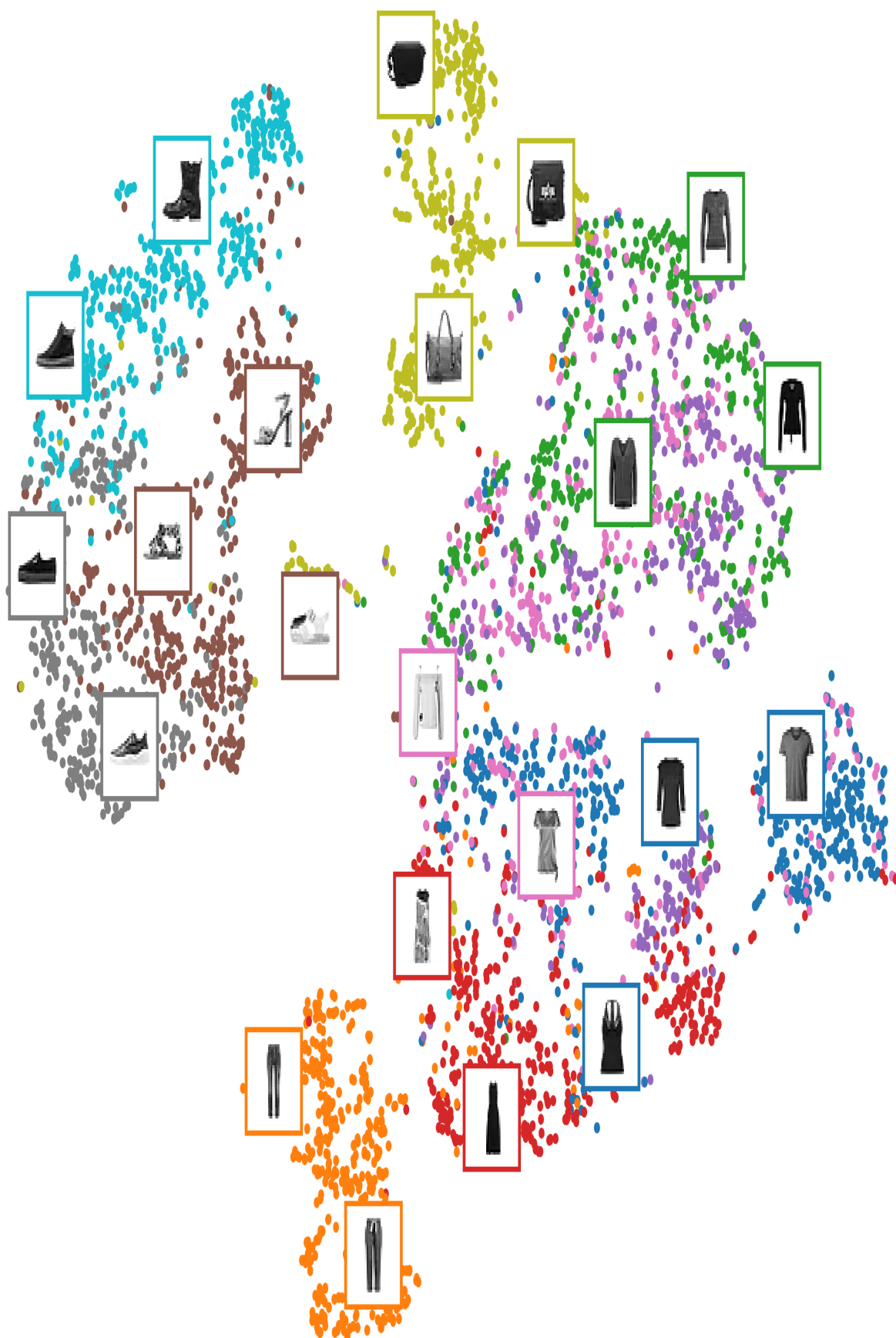


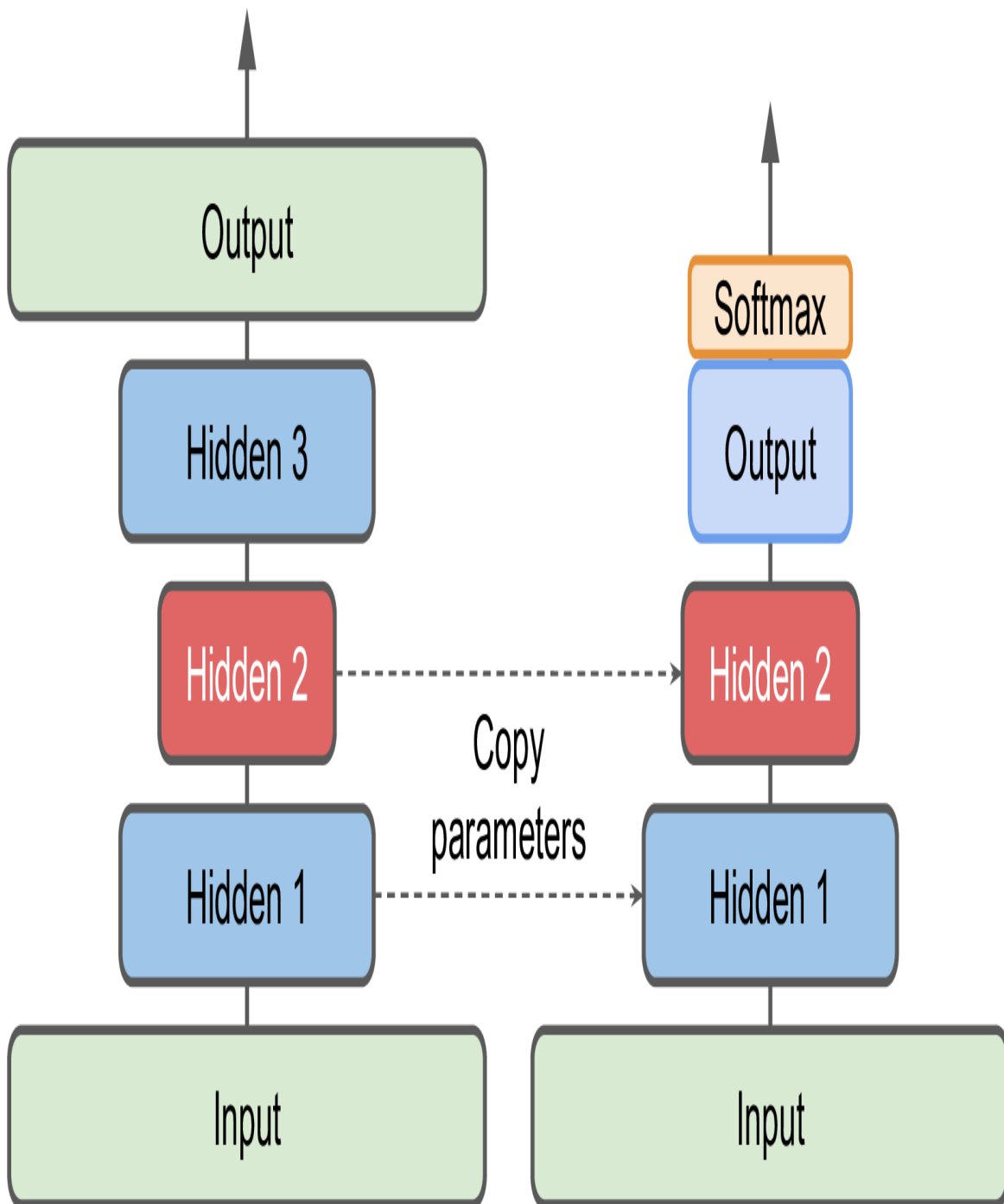
Figure 17-5. Fashion MNIST visualization using an autoencoder followed by t-SNE

So, autoencoders can be used for dimensionality reduction. Another application is for unsupervised pretraining.

Unsupervised Pretraining Using Stacked Autoencoders

As we discussed in [Chapter 11](#), if you are tackling a complex supervised task but you do not have a lot of labeled training data, one solution is to find a neural network that performs a similar task and reuse its lower layers. This makes it possible to train a high-performance model using little training data because your neural network won't have to learn all the low-level features; it will just reuse the feature detectors learned by the existing network.

Similarly, if you have a large dataset but most of it is unlabeled, you can first train a stacked autoencoder using all the data, then reuse the lower layers to create a neural network for your actual task and train it using the labeled data. For example, [Figure 17-6](#) shows how to use a stacked autoencoder to perform unsupervised pretraining for a classification neural network. When training the classifier, if you really don't have much labeled training data, you may want to freeze the pretrained layers (at least the lower ones).



Phase 1

Train the autoencoder
using all the data

Phase 2

Train the classifier
on the labeled data

Figure 17-6. Unsupervised pretraining using autoencoders

NOTE

Having plenty of unlabeled data and little labeled data is common. Building a large unlabeled dataset is often cheap (e.g., a simple script can download millions of images off the internet), but labeling those images (e.g., classifying them as cute or not) can usually be done reliably only by humans. Labeling instances is time-consuming and costly, so it's normal to have only a few thousand human-labeled instances, or even less.

There is nothing special about the implementation: just train an autoencoder using all the training data (labeled plus unlabeled), then reuse its encoder layers to create a new neural network (see the exercises at the end of this chapter for an example).

Next, let's look at a few techniques for training stacked autoencoders.

Tying Weights

When an autoencoder is neatly symmetrical, like the one we just built, a common technique is to *tie* the weights of the decoder layers to the weights of the encoder layers. This halves the number of weights in the model, speeding up training and limiting the risk of overfitting. Specifically, if the autoencoder has a total of N layers (not counting the input layer), and $\mathbf{W}_L$ represents the connection weights of the L^{th} layer (e.g., layer 1 is the first hidden layer, layer $N/2$ is the coding layer, and layer N is the output layer), then the decoder layer weights can be defined as: $\mathbf{W}_L = \mathbf{W}_{N-L+1}^T$ (with $L = N/2+1, \dots, N$).

To tie weights between layers using Keras, let's define a custom layer:

```
class DenseTranspose(tf.keras.layers.Layer):
    def __init__(self, dense, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.dense = dense
        self.activation = tf.keras.activations.get(activation)

    def build(self, batch_input_shape):
```

```

        self.biases = self.add_weight(name="bias",
shape=self.dense.input_shape[-1],
                                     initializer="zeros")
        super().build(batch_input_shape)

    def call(self, inputs):
        Z = tf.matmul(inputs, self.dense.weights[0],
transpose_b=True)
        return self.activation(Z + self.biases)

```

This custom layer acts like a regular Dense layer, but it uses another Dense layer's weights, transposed (setting `transpose_b=True` is equivalent to transposing the second argument, but it's more efficient as it performs the transposition on the fly within the `matmul()` operation). However, it uses its own bias vector. Next, we can build a new stacked autoencoder, much like the previous one, but with the decoder's Dense layers tied to the encoder's Dense layers:

```

dense_1 = tf.keras.layers.Dense(100, activation="relu")
dense_2 = tf.keras.layers.Dense(30, activation="relu")

tied_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    dense_1,
    dense_2
])

tied_decoder = tf.keras.Sequential([
    DenseTranspose(dense_2, activation="relu"),
    DenseTranspose(dense_1),
    tf.keras.layers.Reshape([28, 28])
])

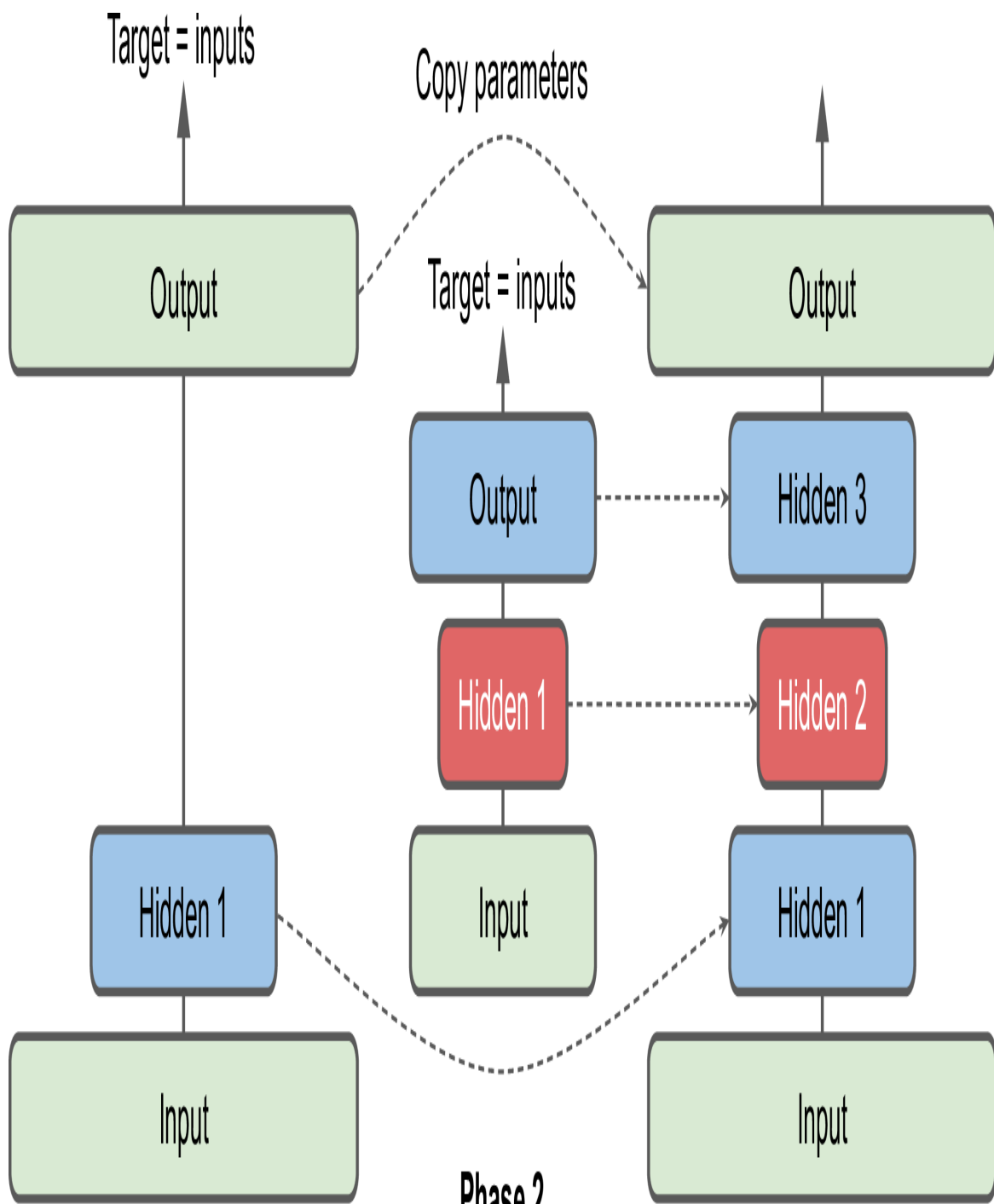
tied_ae = tf.keras.Sequential([tied_encoder, tied_decoder])

```

This model achieves roughly the same reconstruction error as the previous model, using almost half the number of parameters.

Training One Autoencoder at a Time

Rather than training the whole stacked autoencoder in one go like we just did, it is possible to train one shallow autoencoder at a time, then stack all of them into a single stacked autoencoder (hence the name), as shown in **Figure 17-7**. This technique is not used as much these days, but you may still run into papers that talk about “greedy layerwise training,” so it’s good to know what it means.



Phase 1

Train the first autoencoder

Phase 2

Train the second autoencoder
on the training set encoded
by the first encoder

Phase 3

Stack the autoencoders

Figure 17-7. Training one autoencoder at a time

During the first phase of training, the first autoencoder learns to reconstruct the inputs. Then we encode the whole training set using this first autoencoder, and this gives us a new (compressed) training set. We then train a second autoencoder on this new dataset. This is the second phase of training. Finally, we build a big sandwich using all these autoencoders, as shown in **Figure 17-7** (i.e., we first stack the hidden layers of each autoencoder, then the output layers in reverse order). This gives us the final stacked autoencoder (see the “Training One Autoencoder at a Time” section in the notebook for an implementation). We could easily train more autoencoders this way, building a very deep stacked autoencoder.

As we discussed earlier, one of the triggers of the Deep Learning tsunami was the discovery in 2006 by **Geoffrey Hinton et al.** that deep neural networks can be pretrained in an unsupervised fashion, using this greedy layerwise approach. They used restricted Boltzmann machines (RBMs; see <https://hml.info/extra-anns>) for this purpose, but in 2007 **Yoshua Bengio et al. showed²** that autoencoders worked just as well. For several years this was the only efficient way to train deep nets, until many of the techniques introduced in **Chapter 11** made it possible to just train a deep net in one shot.

Autoencoders are not limited to dense networks: you can also build convolutional autoencoders. Let’s look at these now.

Convolutional Autoencoders

If you are dealing with images, then the autoencoders we have seen so far will not work well (unless the images are very small): as we saw in **Chapter 14**, convolutional neural networks are far better suited than dense networks to work with images. So if you want to build an autoencoder for images (e.g., for unsupervised pretraining or dimensionality reduction), you will need to build a *convolutional autoencoder*.³ The encoder is a regular CNN composed of convolutional layers and pooling layers. It typically

reduces the spatial dimensionality of the inputs (i.e., height and width) while increasing the depth (i.e., the number of feature maps). The decoder must do the reverse (upscale the image and reduce its depth back to the original dimensions), and for this you can use transpose convolutional layers (alternatively, you could combine upsampling layers with convolutional layers). Here is a basic convolutional autoencoder for Fashion MNIST:

```
conv_encoder = tf.keras.Sequential([
    tf.keras.layers.Reshape([28, 28, 1]),
    tf.keras.layers.Conv2D(16, 3, padding="same",
activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 14 x 14 x
16
    tf.keras.layers.Conv2D(32, 3, padding="same",
activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 7 x 7 x 32
    tf.keras.layers.Conv2D(64, 3, padding="same",
activation="relu"),
    tf.keras.layers.MaxPool2D(pool_size=2), # output: 3 x 3 x 64
    tf.keras.layers.Conv2D(30, 3, padding="same",
activation="relu"),
    tf.keras.layers.GlobalAvgPool2D() # output: 30
])
conv_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(3 * 3 * 16),
    tf.keras.layers.Reshape((3, 3, 16)),
    tf.keras.layers.Conv2DTranspose(32, 3, strides=2,
activation="relu"),
    tf.keras.layers.Conv2DTranspose(16, 3, strides=2,
padding="same",
activation="relu"),
    tf.keras.layers.Conv2DTranspose(1, 3, strides=2,
padding="same"),
    tf.keras.layers.Reshape([28, 28])
])
conv_ae = tf.keras.Sequential([conv_encoder, conv_decoder])
```

It's also possible to create autoencoders with other architecture types, such as RNNs (see the notebook for an example).

OK, let's step back for a second. So far we have seen various kinds of autoencoders (basic, stacked, and convolutional), and we have looked at

how to train them (either in one shot or layer by layer). We also looked at a couple applications: data visualization and unsupervised pretraining.

Up to now, in order to force the autoencoder to learn interesting features, we have limited the size of the coding layer, making it undercomplete. There are actually many other kinds of constraints that can be used, including ones that allow the coding layer to be just as large as the inputs, or even larger, resulting in an *overcomplete autoencoder*. So let's look at a few more kinds of autoencoders: denoising autoencoders, sparse autoencoders, and variational autoencoders.

Denoising Autoencoders

Another way to force the autoencoder to learn useful features is to add noise to its inputs, training it to recover the original, noise-free inputs. This idea has been around since the 1980s (e.g., it is mentioned in Yann LeCun's 1987 master's thesis). In a [2008 paper](#),⁴ Pascal Vincent et al. showed that autoencoders could also be used for feature extraction. In a [2010 paper](#),⁵ Vincent et al. introduced *stacked denoising autoencoders*.

The noise can be pure Gaussian noise added to the inputs, or it can be randomly switched-off inputs, just like in dropout (introduced in [Chapter 11](#)). [Figure 17-8](#) shows both options.

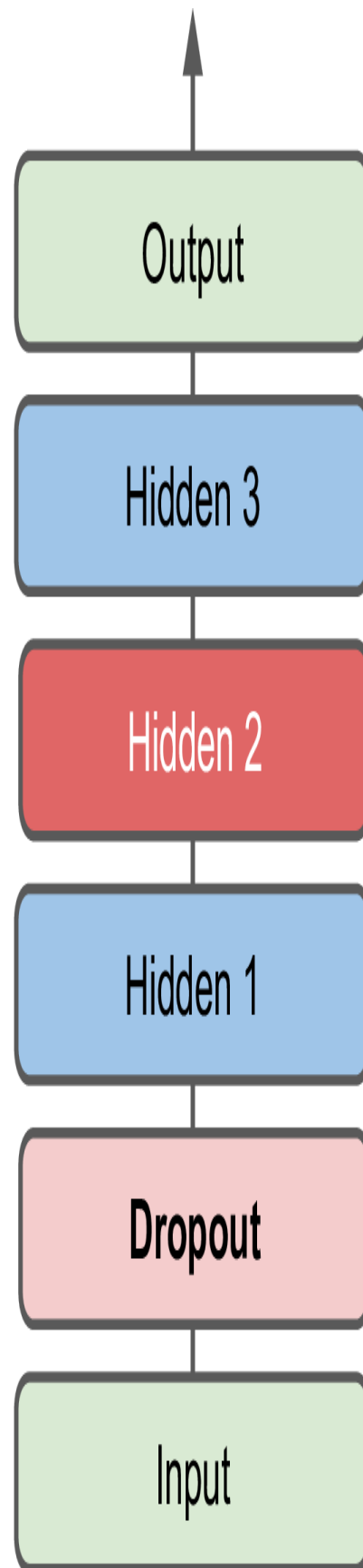
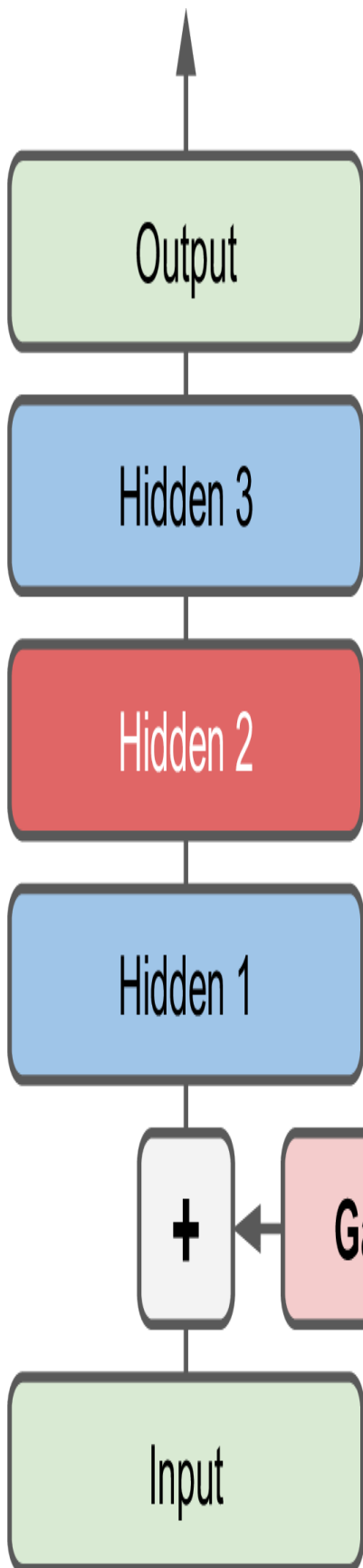


Figure 17-8. Denoising autoencoders, with Gaussian noise (left) or dropout (right)

The implementation is straightforward: it is a regular stacked autoencoder with an additional `Dropout` layer applied to the encoder's inputs (or you could use a `GaussianNoise` layer instead). Recall that the `Dropout` layer is only active during training (and so is the `GaussianNoise` layer):

```
dropout_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(30, activation="relu")
])
dropout_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
dropout_ae = tf.keras.Sequential([dropout_encoder,
    dropout_decoder])
```

Figure 17-9 shows a few noisy images (with half the pixels turned off), and the images reconstructed by the dropout-based denoising autoencoder. Notice how the autoencoder guesses details that are actually not in the input, such as the top of the white shirt (bottom row, fourth image). As you can see, not only can denoising autoencoders be used for data visualization or unsupervised pretraining, like the other autoencoders we've discussed so far, but they can also be used quite simply and efficiently to remove noise from images.



Figure 17-9. Noisy images (top) and their reconstructions (bottom)

Sparse Autoencoders

Another kind of constraint that often leads to good feature extraction is *sparsity*: by adding an appropriate term to the cost function, the autoencoder is pushed to reduce the number of active neurons in the coding layer. For example, it may be pushed to have on average only 5% significantly active neurons in the coding layer. This forces the autoencoder to represent each input as a combination of a small number of activations. As a result, each neuron in the coding layer typically ends up representing a useful feature (if you could speak only a few words per month, you would probably try to make them worth listening to).

A simple approach is to use the sigmoid activation function in the coding layer (to constrain the codings to values between 0 and 1), use a large coding layer (e.g., with 300 units), and add some ℓ_1 regularization to the coding layer's activations. The decoder is just a regular decoder.

```
sparse_l1_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(300, activation="sigmoid"),
    tf.keras.layers.ActivityRegularization(l1=1e-4)
])
sparse_l1_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
sparse_l1_ae = tf.keras.Sequential([sparse_l1_encoder,
sparse_l1_decoder])
```

This `ActivityRegularization` layer just returns its inputs, but as a side effect it adds a training loss equal to the sum of absolute values of its inputs. This only affects training. Equivalently, you could remove the `ActivityRegularization` layer and set `activity_regularizer=tf.keras.regularizers.l1(1e-4)` in the previous layer. This penalty will encourage the neural network to produce codings close to 0, but since it will also be penalized if it does not

reconstruct the inputs correctly, it will have to output at least a few nonzero values. Using the ℓ_1 norm rather than the ℓ_2 norm will push the neural network to preserve the most important codings while eliminating the ones that are not needed for the input image (rather than just reducing all codings).

Another approach, which often yields better results, is to measure the actual sparsity of the coding layer at each training iteration, and penalize the model when the measured sparsity differs from a target sparsity. We do so by computing the average activation of each neuron in the coding layer, over the whole training batch. The batch size must not be too small, or else the mean will not be accurate.

Once we have the mean activation per neuron, we want to penalize the neurons that are too active, or not active enough, by adding a *sparsity loss* to the cost function. For example, if we measure that a neuron has an average activation of 0.3, but the target sparsity is 0.1, it must be penalized to activate less. One approach could be simply adding the squared error $(0.3 - 0.1)^2$ to the cost function, but in practice a better approach is to use the Kullback–Leibler (KL) divergence (briefly discussed in [Chapter 4](#)), which has much stronger gradients than the mean squared error, as you can see in [Figure 17-10](#).

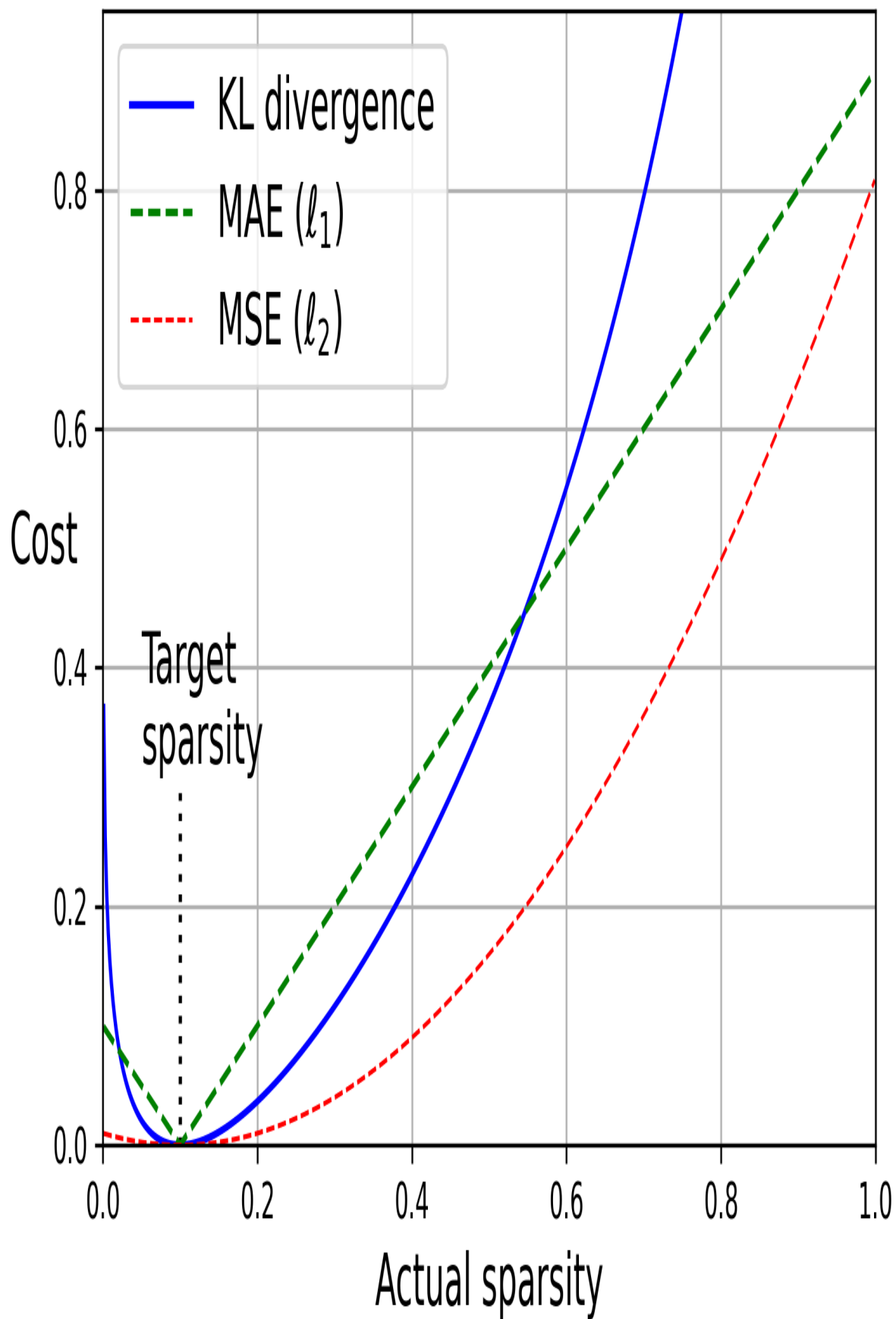


Figure 17-10. Sparsity loss

Given two discrete probability distributions P and Q , the KL divergence between these distributions, noted $D_{\text{KL}}(P \parallel Q)$, can be computed using [Equation 17-1](#).

Equation 17-1. Kullback–Leibler divergence

$$D_{\text{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

In our case, we want to measure the divergence between the target probability p that a neuron in the coding layer will activate and the actual probability q , estimated by measuring the mean activation over the training batch. So the KL divergence simplifies to [Equation 17-2](#).

Equation 17-2. KL divergence between the target sparsity p and the actual sparsity q

$$D_{\text{KL}}(p \parallel q) = p \log \frac{p}{q} + (1 - p) \log \frac{1 - p}{1 - q}$$

Once we have computed the sparsity loss for each neuron in the coding layer, we sum up these losses and add the result to the cost function. In order to control the relative importance of the sparsity loss and the reconstruction loss, we can multiply the sparsity loss by a sparsity weight hyperparameter. If this weight is too high, the model will stick closely to the target sparsity, but it may not reconstruct the inputs properly, making the model useless. Conversely, if it is too low, the model will mostly ignore the sparsity objective and will not learn any interesting features.

We now have all we need to implement a sparse autoencoder based on the KL divergence. First, let's create a custom regularizer to apply KL divergence regularization:

```
kl_divergence = tf.keras.losses.kullback_leibler_divergence

class KLDivergenceRegularizer(tf.keras.regularizers.Regularizer):
    def __init__(self, weight, target):
```

```

        self.weight = weight
        self.target = target

    def __call__(self, inputs):
        mean_activities = tf.reduce_mean(inputs, axis=0)
        return self.weight * (
            kl_divergence(self.target, mean_activities) +
            kl_divergence(1. - self.target, 1. -
mean_activities))

```

Now we can build the sparse autoencoder, using the `KLDivergenceRegularizer` for the coding layer's activations:

```

kld_reg = KLDivergenceRegularizer(weight=5e-3, target=0.1)
sparse_kl_encoder = tf.keras.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(300, activation="sigmoid",
        activity_regularizer=kld_reg)
])
sparse_kl_decoder = tf.keras.Sequential([
    tf.keras.layers.Dense(100, activation="relu"),
    tf.keras.layers.Dense(28 * 28),
    tf.keras.layers.Reshape([28, 28])
])
sparse_kl_ae = tf.keras.Sequential([sparse_kl_encoder,
sparse_kl_decoder])

```

After training this sparse autoencoder on Fashion MNIST, the coding layer will have roughly 10% sparsity.

Variational Autoencoders

An important category of autoencoders was introduced in 2013 by Diederik Kingma and Max Welling and quickly became one of the most popular types of autoencoders: *variational autoencoders* (VAEs).⁶

They are quite different from all the autoencoders we have discussed so far, in these particular ways:

- They are *probabilistic autoencoders*, meaning that their outputs are partly determined by chance, even after training (as opposed to denoising autoencoders, which use randomness only during training).
- Most importantly, they are *generative autoencoders*, meaning that they can generate new instances that look like they were sampled from the training set.

Both these properties make them rather similar to RBMs, but they are easier to train, and the sampling process is much faster (with RBMs you need to wait for the network to stabilize into a “thermal equilibrium” before you can sample a new instance). As their name suggests, variational autoencoders perform variational Bayesian inference, which is an efficient way of carrying out approximate Bayesian inference. Recall that Bayesian inference means updating a probability distribution based on new data, using equations derived from Bayes’ theorem. The original distribution is called the *prior*, while the updated distribution is called the *posterior*. In our case, we want to find a good approximation of the data distribution. Once we have that, we can sample from it.

Let’s take a look at how VAEs work. **Figure 17-11** (left) shows a variational autoencoder. You can recognize the basic structure of all autoencoders, with an encoder followed by a decoder (in this example, they both have two hidden layers), but there is a twist: instead of directly producing a coding for a given input, the encoder produces a *mean coding* μ and a standard deviation σ . The actual coding is then sampled randomly from a Gaussian distribution with mean μ and standard deviation σ . After that the decoder decodes the sampled coding normally. The right part of the diagram shows a training instance going through this autoencoder. First, the encoder produces μ and σ , then a coding is sampled randomly (notice that it is not exactly located at μ), and finally this coding is decoded; the final output resembles the training instance.

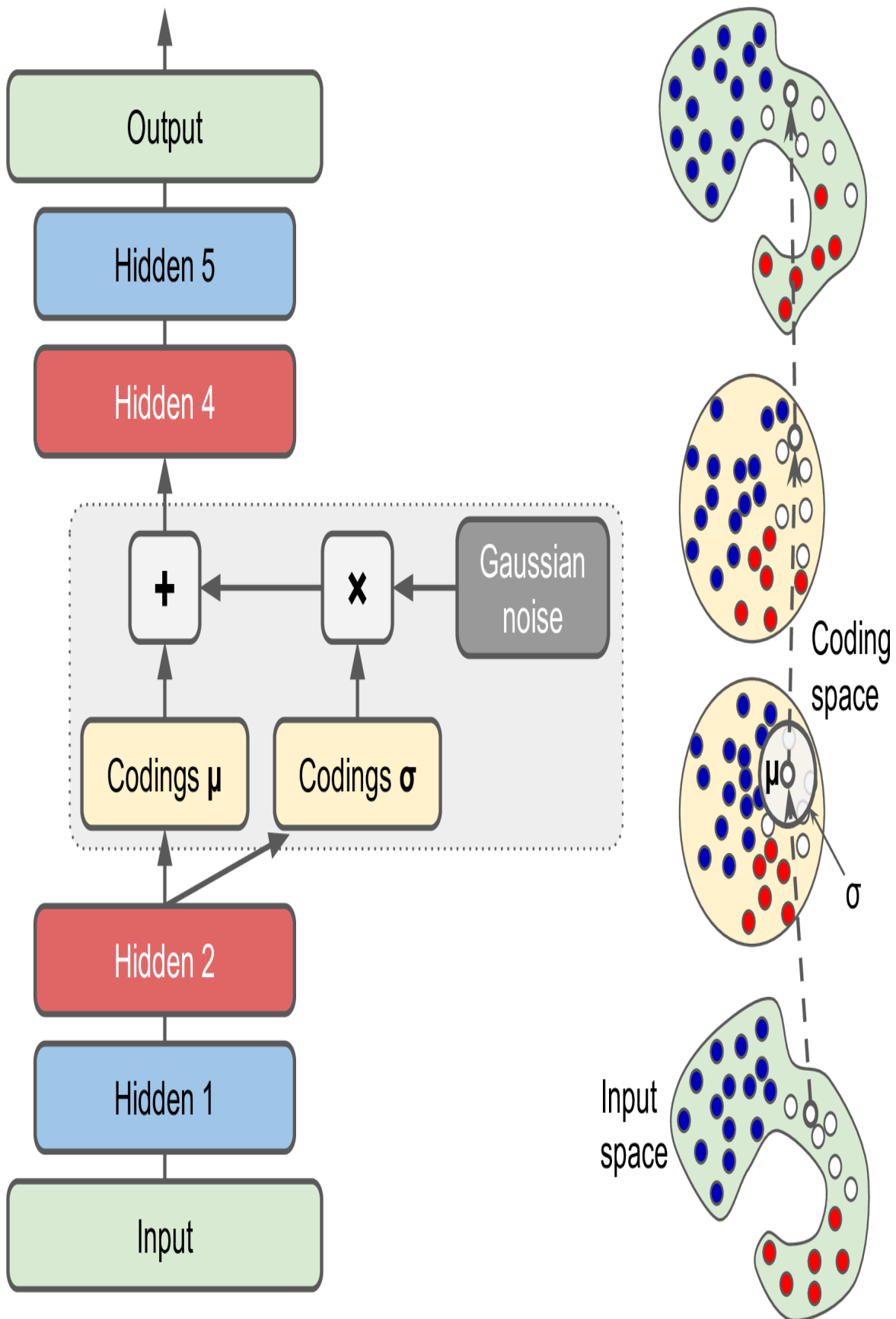


Figure 17-11. Variational autoencoder (left) and an instance going through it (right)

As you can see in the diagram, although the inputs may have a very convoluted distribution, a variational autoencoder tends to produce codings that look as though they were sampled from a simple Gaussian distribution:⁷ during training, the cost function (discussed next) pushes the codings to gradually migrate within the coding space (also called the *latent space*) to end up looking like a cloud of Gaussian points. One great consequence is that after training a variational autoencoder, you can very easily generate a new instance: just sample a random coding from the Gaussian distribution, decode it, and voilà!

Now, let's look at the cost function. It is composed of two parts. The first is the usual reconstruction loss that pushes the autoencoder to reproduce its inputs. We can use the MSE for this, as we did earlier. The second is the *latent loss* that pushes the autoencoder to have codings that look as though they were sampled from a simple Gaussian distribution: it is the KL divergence between the target distribution (i.e., the Gaussian distribution) and the actual distribution of the codings. The math is a bit more complex than with the sparse autoencoder, in particular because of the Gaussian noise, which limits the amount of information that can be transmitted to the coding layer. This pushes the autoencoder to learn useful features. Luckily, the equations simplify, so the latent loss can be computed using **Equation 17-3:**⁸

Equation 17-3. Variational autoencoder's latent loss

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \log(\sigma_i^2) - \sigma_i^2 - \mu_i^2]$$

In this equation, $\mathcal{L}$ is the latent loss, n is the codings' dimensionality, and μ_i and σ_i are the mean and standard deviation of the i^{th} component of the codings. The vectors $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ (which contain all the μ_i and σ_i) are output by the encoder, as shown in **Figure 17-11** (left).

A common tweak to the variational autoencoder's architecture is to make the encoder output $\gamma = \log(\sigma^2)$ rather than σ . The latent loss can then be computed as shown in [Equation 17-4](#). This approach is more numerically stable and speeds up training.

Equation 17-4. Variational autoencoder's latent loss, rewritten using $\gamma = \log(\sigma^2)$

$$\mathcal{L} = -\frac{1}{2} \sum_{i=1}^n [1 + \gamma_i - \exp(\gamma_i) - \mu_i^2]$$

Let's start building a variational autoencoder for Fashion MNIST (as shown in [Figure 17-11](#), but using the γ tweak). First, we will need a custom layer to sample the codings, given μ and γ :

```
class Sampling(tf.keras.layers.Layer):
    def call(self, inputs):
        mean, log_var = inputs
        return tf.random.normal(tf.shape(log_var)) *
        tf.exp(log_var / 2) + mean
```

This `Sampling` layer takes two inputs: `mean` (μ) and `log_var` (γ). It uses the function `tf.random.normal()` to sample a random vector (of the same shape as γ) from the Gaussian distribution, with mean 0 and standard deviation 1. Then it multiplies it by `exp( $\gamma$  / 2)` (which is equal to σ , as you can verify mathematically), and finally it adds μ and returns the result. This samples a codings vector from the Gaussian distribution with mean μ and standard deviation σ .

Next, we can create the encoder, using the Functional API because the model is not entirely sequential:

```
codings_size = 10

inputs = tf.keras.layers.Input(shape=[28, 28])
Z = tf.keras.layers.Flatten()(inputs)
Z = tf.keras.layers.Dense(150, activation="relu")(Z)
Z = tf.keras.layers.Dense(100, activation="relu")(Z)
codings_mean = tf.keras.layers.Dense(codings_size)(Z) #  $\mu$ 
codings_log_var = tf.keras.layers.Dense(codings_size)(Z) #  $\gamma$ 
```